Department Of Computer Studies

B. Sc. (CA & IT) Honours- III

U13A2OOP- OBJECT ORIENTED CONCEPTS AND PROGRAMMING

Unit2: - Overview of JAVA

Introduction to Java

- Java is a purely an object – oriented language.
- In 1991, JamesGosling and Patrick Naughton developed named "OAK" at Sun Microsystems.
- Java is simple and platform-independent.
- In 1995, this language was renamed as "Java".
- Java history is interesting to know. The history of java starts from Green
- Java team members (also known as Green Team), initiated a revolutionary task to develop a language for digital devices such as set-top boxes, televisions etc.
- For the green team members, it was an advance concept at that time. But it was suited for internet programming. Later, Java technology as incorporated by Netscape.

Introduction to Java – History

- There are given the major points that describes the history of java.
- James Gosling, Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991. The small team of sun engineers called Green Team.
- Originally designed for small, embedded systems in electronic appliances like set-top boxes.
- Firstly, it was called "Greentalk" by James Gosling
- After that, it was called Oak and was developed as a part of the green project.
- Why Oak? Oak is a symbol of strength and choosen as a national tree of many countries like U.S.A.,France, Germany, Romania etc.
- In 1995, Oak was renamed as

"Java" because it was already a trademark by Oak Technologies.

- Java is an island of Indonesia where first coffee was produced called java coffee.
- Notice that Java is just a name not an acronym.
- Originally developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and released in 1995.
- In 1995, Time magazine called Java one of the Ten Best Products of 1995. JDK 1.0 released in (January 23, 1996).

Introduction to Java – Versions

- Sun Microsystems has been releasing new versions of JDK. The originally version of Java released in 1995, was called JDK1.0 version.
- The versions are JDK1.1, JDK1.2, JDK1.3, JDK1.4 and JDK1.5, JDK1.6, JDK1.7 and JDK1.8.

JDK

- Java Development Kit (JDK) comes with a collection of tools that are used for developing and running java-based programs.
- The JDK is used to compile Java applications and applets.
- JDK needs more Disk space as it contains JRE along with various development tools.
- It includes JRE, set of API classes, Java compiler, Web start and additional files needed to write Java applets and applications.

JRE

- JRE is Java Runtime Environment which is required to run Java Applications also. JRE alone does not contains compiler etc for Java Development.
- The java programming language adds the portability by converting the source code to byte code version which can be interpreted by the JRE and gets converted to the platform specific executable ones.
- Thus, for different platforms one has corresponding implementation of JRE. But JRE has to meet the specification JVM (Java Virtual Machine) Concept that serves as a link between the Java libraries and the platform specific implementation of JRE.
- Thus, JVM helps in the abstraction of inner implementation from the programmers who make use of libraries for their programs.
- JRE is a subset of JDK. - Two steps for a java program

- ie.,compile and interpret. - JDK does compilation but JRE can't. - Both JRE and JDK does interpretation

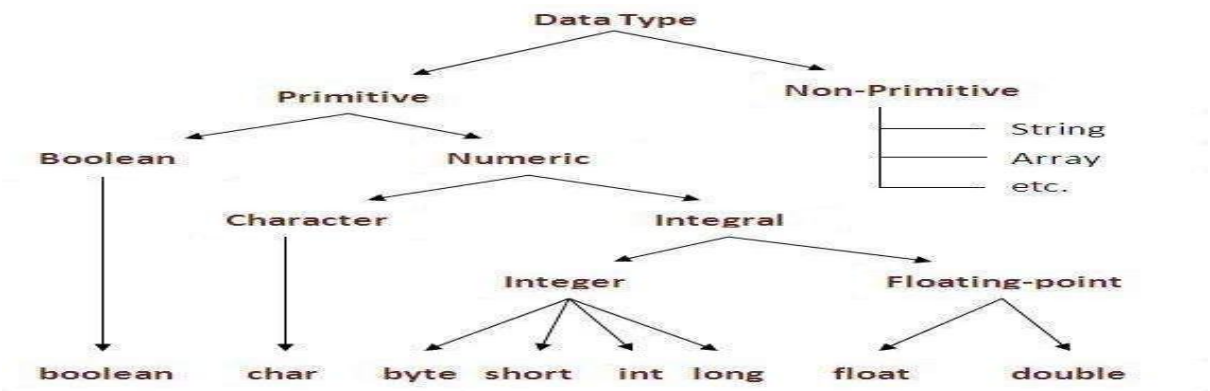
JVM

- JVM (Java Virtual Machine) is machine dependent.
- JVM is virtual because it is generally implemented in software on a top of a 'real' hardware platform and Operating System ' All Java Programs are compiled for the JVM '
- The use of the JVM is simple: - To change the byte code into the machine specific code. So, the JVM converts the byte code (i.e the code which we get when we compile the .java class) into the code that your machine/OS understands. '
- JVM is machine specific. So JVM for windows will be different from JVM for Mac or Unix. '
- JVM is one of the most important part in the java engine.
- It is due to JVM only that we say java code is "Compile Once, Run Anywhere" programming language.

Bytecode, JIT

- A Java program is first compiled into the machine code that is understood by JVM. Such a code is called Byte Code.
- Although the details of the JVM will differ from platform to platform, they all interpret the Java Byte Code into executable native machine code.
- The Just in Time (JIT) compiler compiles the byte code into executable machine code in real time, on a piece-by-piece demand basis.

DATA TYPE IN JAVA



Data Types – Default Values

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Data types are divided into two groups:

- Primitive data types - includes byte, short, int, long, float, double, boolean and char
- Non-primitive data types - such as String, Arrays and Classes.

A primitive data type specifies the size and type of variable values, and it has no additional methods.

Numbers

Primitive number types are divided into two groups:

Integer types store whole numbers, positive or negative (such as 123 or -456), without decimals. Valid types are byte, short, int and long. Which type you should use, depends on the numeric value.

Floating point types represents numbers with a fractional part, containing one or more decimals. There are two types: float and double.

Boolean Types

Very often in programming, you will need a data type that can only have one of two values, like:

- YES / NO
- ON / OFF
- TRUE / FALSE

Characters

The char data type is used to store a single character. The character must be surrounded by single quotes, like 'A' or 'c':

Strings

The String data type is used to store a sequence of characters (text). String values must be surrounded by double quotes:

Non-Primitive Data Types

Non-primitive data types are called reference types because they refer to objects.

Examples of non-primitive types are Strings, Arrays, Classes, Interface, etc.

Variables in Java

A variable is a named memory location used to store data. The value stored in a variable can be changed during program execution.

A variable is a container that holds data values during the execution of a Java program.

Syntax of Variable Declaration

dataType variableName;

int number;

EXAMPLE

```
public class VariableDemo {
   public static void main(String args[]) {
      int age = 22;
      String name = "Rahul";
      double salary = 25000.50;
      System.out.println("Name : " + name);
      System.out.println("Age : " + age);
      System.out.println("Salary : " + salary);
   }
}
```

Types of Variables in Java

Java has 3 types of variables:

1. Local Variable
2. Instance Variable
3. Static Variable

1. Local Variable

A local variable is declared inside a method, constructor, or block.

Characteristics

- Declared inside a method
- Accessible only within that method
- Must be initialized before use

```
public class LocalVariable {
   public static void main(String args[]) {
      int number = 100;
      System.out.println(number);
   }
}
```

2. Instance Variable

Instance variables are declared inside a class but outside methods.

```
class Student {
   String name = "Raj";
   int age = 20;
   void display() {
      System.out.println(name);
      System.out.println(age);
   }
   public static void main(String args[]) {
      Student s = new Student();
      s.display();
   }
```

```
}
```

3. Static Variable

A static variable belongs to the class, not individual objects.

```
class College {
   static String collegeName = "ABC College";
   void display() {
      System.out.println(collegeName);
   }
   public static void main(String args[]) {
      College s1 = new College();
            s1.display();
  }
}
```

Java Operators

Operators are used to perform operations on variables and values.

Java divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y

-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Java Assignment Operators

Assignment operators are used to assign values to variables.

The **addition assignment** operator (+=) adds a value to a variable.

A list of all assignment operators:

Operator	**Example**	**Same As**
=	x = 5	x = 5
+=	x += 3	x = x + 3

-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Java Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming, because it helps us to find answers and make decisions.

The return value of a comparison is either true or false. These values are known as *Boolean values*

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Java Logical Operators

You can also test for true or false values with logical operators.

Logical operators are used to determine the logic between variables or values

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10

||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

```
public class allop {

   public static void main(String[] args) {

      int a = 20;

      int b = 10;

      boolean x = true;

      boolean y = false;

      // ================================

      // Arithmetic Operators

      // ================================

      System.out.println("===== Arithmetic Operators =====");

      System.out.println("a + b = " + (a + b));

      System.out.println("a - b = " + (a - b));

      System.out.println("a * b = " + (a * b));

      System.out.println("a / b = " + (a / b));

      System.out.println("a % b = " + (a % b));

      // ================================

      // Assignment Operators

      // ================================
```

```
System.out.println("\n===== Assignment Operators =====");

int c = 10;

System.out.println("Initial Value = " + c);

c += 5;

System.out.println("After += : " + c);

c -= 3;

System.out.println("After -= : " + c);

c *= 2;

System.out.println("After *= : " + c);

c /= 4;

System.out.println("After /= : " + c);

c %= 3;

System.out.println("After %= : " + c);

// ==============================

// Comparison Operators

// ==============================

System.out.println("\n===== Comparison Operators =====");

System.out.println("a == b : " + (a == b));

System.out.println("a != b : " + (a != b));

System.out.println("a > b  : " + (a > b));

System.out.println("a < b  : " + (a < b));

System.out.println("a >= b : " + (a >= b));

System.out.println("a <= b : " + (a <= b));
```

```
        // ==============================

        // Logical Operators

        // ==============================

        System.out.println("\n===== Logical Operators =====");

        System.out.println("x && y = " + (x && y));

        System.out.println("x || y = " + (x || y));

        System.out.println("!x = " + (!x));

        System.out.println("!y = " + (!y));

          }

}
```

Type conversion and casting: -

Type casting is when you assign a value of one primitive data type to another type.

In Java, there are two types of casting:

- **Widening Casting** (Implicit) - converting a smaller type to a larger type size
  byte -> short -> char -> int -> long -> float -> double
- **Narrowing Casting** (Explicit) - converting a larger type to a smaller size type
  double -> float -> long -> int -> char -> short -> byte

Widening Casting

Widening casting is done automatically when passing a smaller size type to a larger size type.

```
public class wide {
   public static void main(String args[]) {
      int number = 100;
      double value = number;
      System.out.println("Integer Value = " + number);
      System.out.println("Double Value = " + value);
   }
```

}**Narrowing Casting**

Narrowing casting must be done manually by placing the type in parentheses in front of the value.

```
public class narrow {
   public static void main(String args[]) {
      double number = 95.75;
      int value = (int) number;
      System.out.println("Double = " + number);
      System.out.println("Integer = " + value);
   }
}
```

Java Control Statements | Control Flow in Java

Java provides three types of control flow statements.

1. Decision Making statements
    - if statements
    - switch statement
2. Loop statements
    - do while loop
    - while loop
    - for loop
    - for-each loop
3. Jump statements
    - break statement
    - continue statement

Decision-Making statements:

1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false.

1. Simple if statement
2. if-else statement
3. if-else-if ladder

1) Simple if statement:

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

```
1. if(condition) {
2. statement 1; //executes when condition is true
3. }
```

2) if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

Syntax:

```
1. if(condition) {
2. statement 1; //executes when condition is true
3. }
4. else{
5. statement 2; //executes when condition is false
6. }
```

3) if-else-if ladder

The if-else-if statement contains the if-statement followed by multiple else-if statements.

Syntax of if-else-if statement is given below.

```
if(condition 1) {
statement 1; //executes when condition 1 is true
}
else if(condition 2) {
statement 2; //executes when condition 2 is true
}
else {
statement 2; //executes when all the conditions are false
}
```

Switch Statement:

The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

he syntax to use the switch statement is given below.

```
switch (expression){
    case value1:
     statement1;
     break;
    .
    .
    .
    case valueN:
     statementN;
     break;
    default:
     default statement;
}
```

```
public class allde{

  public static void main(String[] args) {

    int age = 20;

    int number = 15;

    int marks = 82;

    boolean license = true;
```

```
int day = 3;
// ============================
// 1. if Statement
// ============================
System.out.println("===== IF Statement =====");
if(age >= 18)
{
   System.out.println("Eligible for Voting");
}
// ============================
// 2. if-else Statement
// ============================
System.out.println("\n===== IF-ELSE Statement =====");
if(number % 2 == 0)
{
   System.out.println(number + " is Even");
}
else
{
   System.out.println(number + " is Odd");
}
// ============================
// 3. else-if Ladder
// ============================
System.out.println("\n===== ELSE-IF Ladder =====");
if(marks >= 90)
{
   System.out.println("Grade A");
```

```
}
else if(marks >= 75)
{
   System.out.println("Grade B");
}
else if(marks >= 50)
{
   System.out.println("Grade C");
}
else
{
   System.out.println("Fail");
}
// ============================
// 4. Nested if
// ============================
System.out.println("\n===== Nested IF =====");
if(age >= 18)
{
   if(license)
   {
      System.out.println("Eligible to Drive");
   }
   else
   {
      System.out.println("Driving License Required");
   }
}
```

```
else
{
   System.out.println("Not Eligible");
}
// ============================
// 5. Switch Statement
// ============================
System.out.println("\n===== Switch Statement =====");
switch(day)
{
   case 1:
      System.out.println("Monday");
      break;
   case 2:
      System.out.println("Tuesday");
      break;
   case 3:
      System.out.println("Wednesday");
      break;
   case 4:
      System.out.println("Thursday");
      break;
   case 5:
      System.out.println("Friday");
      break;
   case 6:
      System.out.println("Saturday");
      break;
```

```
        case 7:
            System.out.println("Sunday");
            break;
         default:
            System.out.println("Invalid Day");
      }
   }
}
```

Loop Statements: -

Types of Loops in Java

Java mainly provides three types of loops:

1. for Loop
2. while Loop
3. do-while Loop

1. for Loop

Definition

The for loop is used when the number of iterations is known in advance.

Syntax

```
for(initialization; condition; increment/decrement)
{
   // Statements
}
```

2. while Loop

Definition

The while loop executes statements as long as the condition is true.

Syntax

```
while(condition)
{
   // Statements
}
```

3. do-while Loop

Definition

The do-while loop executes the block of code at least once, even if the condition is false.

Syntax

```
do
{
   // Statements
}
while(condition);
```

EXAMPLE

```
public class allloop{
   public static void main(String[] args) {
      // ====================================
      // 1. FOR LOOP
      // ====================================
      System.out.println("===== FOR LOOP =====");
      for(int i = 1; i <= 5; i++)
      {
         System.out.println(i);
      }
      // ====================================
      // 2. WHILE LOOP
      // ====================================
      System.out.println("\n===== WHILE LOOP =====");
      int i = 1;
```

```
while(i <= 5)
{
   System.out.println(i);
   i++;
}
// ====================================
// 3. DO-WHILE LOOP
// ====================================
System.out.println("\n===== DO-WHILE LOOP =====");
int j = 1;
do
{
   System.out.println(j);
   j++;
}
while(j <= 5);
// ====================================
// 4. NESTED LOOP
// ====================================
System.out.println("\n===== NESTED LOOP =====");
for(int row = 1; row <= 3; row++)
{
   for(int col = 1; col <= 3; col++)
   {
      System.out.print("* ");
   }
   System.out.println();
}
```

```
        // ====================================
        // 5. BREAK STATEMENT
        // ====================================
        System.out.println("\n===== BREAK STATEMENT =====");
        for(int k = 1; k <= 10; k++)
        {
            if(k == 5)
            {
                break;
            }
            System.out.println(k);
        }
        // ====================================
        // 6. CONTINUE STATEMENT
        // ====================================
        System.out.println("\n===== CONTINUE STATEMENT =====");
        for(int n = 1; n <= 5; n++)
        {
            if(n == 3)
            {
                continue;
            }
            System.out.println(n);
        }
    }
}
```

Java Garbage Collection

Garbage Collection (GC) is an automatic memory management process in Java.

When an object is no longer used by a program, Java automatically removes that object from memory. This process is called **Garbage Collection**.

Simple Example

Student s = new Student();

s = null;

After s = null, the Student object no longer has a reference. Therefore, it becomes eligible for Garbage Collection.

Java programmers do not normally need to manually delete objects.

Advantage of Garbage Collection

- It makes java memory efficient because garbage collector removes the unreferenced objects from heap memory.
- It is automatically done by the garbage collector(a part of JVM) so we don't need to make extra efforts.

Using System.gc()
It requests the JVM to perform Garbage Collection.

Important

System.gc() does **not guarantee** that Garbage Collection will happen immediately.

The JVM decides when to actually run the Garbage Collector.

System.gc();

Example:-

```
public class GarbageCollectionDemo {
   public static void main(String[] args) {
      GarbageCollectionDemo obj1 =
            new GarbageCollectionDemo();
      GarbageCollectionDemo obj2 =
            new GarbageCollectionDemo();
      obj1 = null;
      obj2 = null;
      System.gc();
```

```
        System.out.println(
                "Garbage Collection Requested"
        );
    }
}
```

Garbage Collection Using finalize()

The finalize() method was historically used to perform cleanup before an object was removed by the Garbage Collector.

Java static keyword

The **static keyword** in Java is used for **memory management mainly**. A static member is created only **once** in memory and is shared by all objects of the class.

Syntax

static dataType variableName;

Example:

static int count;

Types of Static Members

Java mainly provides:

1. Static Variable
2. Static Method
3. Static Block
4. Static Nested Class

1. Static Variable

A variable declared with the static keyword is called a **static variable** or **class variable**.

```
class Static1 {
    int rollNo;
    String name;
    static String college = "DCS College";
    Static1(int r, String n) {
```

```
        rollNo = r;
        name = n;
    }
    void display() {
        System.out.println(
                rollNo + " " + name + " " + college
        );
    }
    public static void main(String[] args) {
        Static1 s1 = new Static1(101, "Raj");
        Static1 s2 = new Static1(102, "Priya");
        s1.display();
        s2.display();
    }
}
```

2. Static Method

A method declared with the static keyword is called a **static method**.

A static method can be called without creating an object.

Syntax

```
ClassName.methodName();
```

```
class Calculator1 {
    static void add(int a, int b) {
        System.out.println(
                "Addition = " + (a + b)
```

```
        );
    }
    public static void main(String[] args) {
        Calculator1.add(10, 20);
    }
}
```

3. Static Block

A static block is used to initialize static variables.

It executes **only once when the class is loaded into memory**.

Syntax

```
static
{
    // Statements
}
```

Example:-

```
class StaticBlockDemo {
    static int number;
    static
    {
        number = 100;
        System.out.println(
                "Static Block Executed"
        );
    }
```

```
    public static void main(String[] args) {
        System.out.println(
                "Number = " + number
        );
    }
}
```

Java final keyword:

It can be applied to:

1. **Variables**
2. **Methods**
3. **Classes**

Simple Meaning

Used With	Effect
final Variable	Value cannot be changed
final Method	Cannot be overridden
final Class	Cannot be inherited

1. final Variable

A final variable can be assigned only once.

Syntax

```
final dataType VARIABLE_NAME = value;
```

Example

```
final int MAX_MARKS = 100;
```

After assigning the value, we cannot change it.

```
public class FinalVariableDemo {
    public static void main(String[] args) {
        final int MAX_MARKS = 100;
```

```
        System.out.println("Maximum Marks: " + MAX_MARKS);
        // MAX_MARKS = 200;  // Compilation Error
    }
}
```

2. Final Method

A method declared using final cannot be overridden by a child class.

```
class Parent {
    final void display() {
        System.out.println(
            "This is a final method"
        );
    }
}
```

A child class cannot override it:

```
class Child extends Parent {
    // Error
    // void display()
    // {
    // }
}
```

3. Final Class

A class declared using final cannot be inherited.

Example

```
final class Vehicle {
```

```
    void display() {
        System.out.println(
                "Vehicle Class"
        );
    }
}
```

The following is not allowed:

```
class Car extends Vehicle {
    // Error
}
```

Constructor in Java

A **constructor** is a special member of a class that is used to **initialize objects**.

When an object is created using the new keyword, the constructor is automatically called.

Example

```
Student s = new Student();
```

Here:

Student()

is the constructor.

Important Characteristics of a Constructor

1. The constructor name must be the same as the class name.
2. A constructor does not have a return type.
3. It is automatically called when an object is created.
4. Constructors are used to initialize object data.
5. A class can have multiple constructors.
6. Constructors support **constructor overloading**.

Basic Syntax

```
class ClassName {
   ClassName() {
      // Constructor body
   }
}
```

Example:-

Default Constructor

A constructor that does not have any parameters is called a **default constructor** or **no-argument constructor**.

```
class Student12 {
   Student12() {
      System.out.println("Student Object Created");
   }
   public static void main(String[] args) {
      Student12 s = new Student12();
   }
}
```

Parameterized Constructor

A constructor that accepts parameters is called a **parameterized constructor**.

```
class Student123 {
   int rollNo;
   String name;
   Student123(int r, String n) {
      rollNo = r;
      name = n;
```

```
    }
    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
    public static void main(String[] args) {
        Student123 s =
                new Student123(101, "Raj");
        s.display();
    }
}
```

Constructor Overloading

When a class has more than one constructor with different parameter lists, it is called **constructor overloading**.

```
class Student1234 {
    int rollNo;
    String name;
    // No-argument constructor
    Student1234() {
        rollNo = 0;
        name = "Ram";
    }
    // One-parameter constructor
    Student1234(int rollNo) {
```

```
        this.rollNo = rollNo;
        name = "Laxman";
    }
    // Two-parameter constructor
    Student1234(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }
   void display() {
        System.out.println(
                rollNo + " " + name
        );
    }
    public static void main(String[] args) {
        Student1234 s1 = new Student1234();
        Student1234 s2 = new Student1234(101);
        Student1234 s3 = new Student1234(102, "Rahul");
        s1.display();
        s2.display();
        s3.display();
    }
}
```

Destructor

A destructor is a special method used to perform cleanup before an object is destroyed.

java uses an automatic Garbage Collector to manage unused objects and memory.

Access Specifiers

Access Specifiers, also called **Access Modifiers**, define the accessibility or visibility of classes, variables, methods, and constructors in Java.

They control **where a class member can be accessed**.

Java provides four access levels:

1. private
2. default (no keyword)
3. protected
4. public

2. Types of Access Specifiers

Access Specifier	Same Class	Same Package	Subclass in Different Package	Other Package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓*	✗
public	✓	✓	✓	✓

private Access Specifier

A private member can be accessed **only inside the same class**.

Default Access Specifier

When no access specifier is written, Java uses **default access**.

protected Access Specifier

A protected member can be accessed:

- Inside the same class.
- By classes in the same package.
- By child classes in another package through inheritance.

public Access Specifier

A public member can be accessed from anywhere.

```
class AccessDemo {
    // Private Member
    private int privateNumber = 10;
    // Default Member
    int defaultNumber = 20;
    // Protected Member
    protected int protectedNumber = 30;
    // Public Member
    public int publicNumber = 40;
    // Private Method
    private void privateMethod() {
        System.out.println(
                "Private Method"
        );
    }
    // Default Method
    void defaultMethod() {
        System.out.println(
                "Default Method"
        );
    }
    // Protected Method
    protected void protectedMethod() {
        System.out.println(
```

```
            "Protected Method"
        );
    }
    // Public Method
    public void publicMethod() {
        System.out.println(
            "Public Method"
        );
    }
    public static void main(String[] args) {
        AccessDemo obj =
            new AccessDemo();
        // Private Access
        System.out.println(
            "Private Variable: "
            + obj.privateNumber
        );
        obj.privateMethod();
        // Default Access
        System.out.println(
            "Default Variable: "
            + obj.defaultNumber
        );
        obj.defaultMethod();
```

```
        // Protected Access
        System.out.println(
            "Protected Variable: "
            + obj.protectedNumber
        );
        obj.protectedMethod();
        // Public Access
        System.out.println(
            "Public Variable: "
            + obj.publicNumber
        );
        obj.publicMethod();
    }
}
```

Friend Function in Java

- Java does not support Friend Functions.
- The concept of Friend Function exists in C++, but Java has no friend keyword or friend function.
- A Friend Function is a function that is not a member of a class but **can access the private and protected members of that class.**
- It is declared using the friend keyword.
- Java provides better security using Encapsulation.

Instead of friend functions, Java uses:

- Getter methods
- Setter methods
- Public methods

Java this Keyword

- This keyword in Java is a reference variable that refers to the **current object** of the class.

- It is used to distinguish between instance variables and local variables when they have the same name.

Syntax

this.variableName;

or

this.methodName();

```
class stud1 {

   int rollNo;

  stud1(int rollNo) {

      this.rollNo = rollNo;
   }
   void display() {

      System.out.println("Roll No = " + rollNo);
   }
   public static void main(String[] args) {
      stud1 s = new stud1(101);
      s.display();
   }
}
```

Array in java

An array in Java is a collection of elements of the same data type stored under a single variable name.

Advantages

- Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.
- Random access: We can get any data located at an index position.

Disadvantages

- Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

int marks[] = new int[50];

Array Index

Consider:

int marks[] = {80, 75, 90, 85, 70};

The first element is always at index 0.

Array Declaration

There are two common ways to declare an array.

Method 1

int marks[];

Method 2

int[] marks;

Array Creation

After declaration, we can create an array using new.

int[] marks;

marks = new int[5];

This creates an array capable of storing **5 integers**.

Declaration and Creation Together

int[] marks = new int[5];

This creates an integer array containing five elements.

Accessing Array Elements

EXAMPLE: -

public class Main {

```
  public static void main(String[] args) {
    String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
    System.out.println(cars[0]);
  }
}
```

Change an Array Element

```
public class Main {
  public static void main(String[] args) {
    String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
    cars[0] = "Opel";
    System.out.println(cars[0]);
  }
}
```

Array Length

```
public class Main {
  public static void main(String[] args) {
    String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
    System.out.println(cars.length);
  }
}
```

The new Keyword

You can also create an array by specifying its size with new. This makes an empty array with space for a fixed number of elements.

Example:-

```
public class Main {
```

```
  public static void main(String[] args) {
    String[] cars = new String[4]; // size is 4
    cars[0] = "Volvo";
    cars[1] = "BMW";
    cars[2] = "Ford";
    cars[3] = "Mazda";
    System.out.println(cars[0]); // Outputs Volvo
  }
}
```

Java Arrays Loop

You can loop through the array elements with the for loop, and use the length property to specify how many times the loop should run.

Example

```
public class Main {
  public static void main(String[] args) {
    String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
    for (int i = 0; i < cars.length; i++) {
      System.out.println(cars[i]);
    }
  }
}
```

Calculate the Sum of Elements

```
public class Main {
  public static void main(String[] args) {
    int[] numbers = {1, 5, 10, 25};
```

```
    int sum = 0;
    // Loop through the array and add each element to sum
    for (int i = 0; i < numbers.length; i++) {
      sum += numbers[i];
    }
    System.out.println("The sum is: " + sum);
  }
}
```

In Java, arrays are mainly classified based on the **number of dimensions** they have.

1. **One-Dimensional Array**
2. **Multidimensional Array**

1. One-Dimensional Array

A one-dimensional array stores elements in a single sequence or list.

Practical

```
public class OneDArray {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("One-Dimensional Array:");
        for (int i = 0; i < numbers.length; i++) {
            System.out.println(numbers[i]);
        }
    }
}
```

2. Multidimensional Array

A **multidimensional array** has **more than two dimensions**.

You can use it to store data in a table with rows and columns.

To create a two-dimensional array, write each row inside its own **curly braces**:

int[][] myNumbers = { {1, 4, 2}, {3, 6, 8} };

Here, **myNumbers** has two arrays (two rows):

- First row: {1, 4, 2}
- Second row: {3, 6, 8}

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Example :-

```
public class Main {
  public static void main(String[] args) {
    int[][] myNumbers = { {1, 4, 2}, {3, 6, 8, 5, 2} };
    for (int row = 0; row < myNumbers.length; row++) {
      for (int col = 0; col < myNumbers[row].length; col++) {
        System.out.println("myNumbers[" + row + "][" + col + "] = " + myNumbers[row][col]);
      }
    }
  }
}
```

Output

```
myNumbers[0][0] = 1
myNumbers[0][1] = 4
```

myNumbers[0][2] = 2
myNumbers[1][0] = 3
myNumbers[1][1] = 6
myNumbers[1][2] = 8
myNumbers[1][3] = 5
myNumbers[1][4] = 2

String Class and StringBuffer Class in Java:-

String is immutable (cannot be changed after creation).

StringBuffer is mutable (can be changed after creation).

1. String Class

Definition

A String is a sequence of characters enclosed within double quotation marks (").

Syntax

String variableName = "Hello";

Example:-

```
public class StringDemo {

   public static void main(String[] args) {

      String name = "Raj";

      System.out.println(name);

   }

}
```

Common String Methods

Method	Description	Example
length()	Returns length of string	str.length()
charAt()	Returns character at index	str.charAt(2)
concat()	Joins two strings	str.concat("Java")
equals()	Compares two strings	str.equals(str2)
toUpperCase()	Converts to uppercase	str.toUpperCase()

Method	Description	Example
toLowerCase()	Converts to lowercase	str.toLowerCase()
substring()	Returns part of string	str.substring(2,5)
replace()	Replaces characters	str.replace('a','o')
indexOf()	Finds index of character	str.indexOf('a')
trim()	Removes spaces	str.trim()

Example 1: length()

```
public class StringLength {
   public static void main(String[] args) {
      String str = "Programming";
      System.out.println("Length = " + str.length());
   }
}
```

Output

```
Length = 11
```

Example 2: charAt()

```
public class StringChar {
   public static void main(String[] args) {
      String str = "Java";
      System.out.println(str.charAt(0));
      System.out.println(str.charAt(2));
   }
}
```

Output

```
J
V
```

Example 3: concat()

```
public class StringConcat {
   public static void main(String[] args) {
      String first = "Java";
      String second = " Programming";
      String result = first.concat(second);
      System.out.println(result);
   }
}
```

Output

```
Java Programming
```

Example 4: equals()

```
public class StringEquals {
   public static void main(String[] args) {
      String s1 = "Java";
      String s2 = "Java";
      System.out.println(s1.equals(s2));
   }
}
```

Output

```
True
```

Example 5: toUpperCase()

```
public class UpperCaseDemo {
   public static void main(String[] args) {
```

```
        String name = "java";
        System.out.println(name.toUpperCase());
    }
}
```

Output

```
JAVA
```

Example 6: toLowerCase()

```
public class LowerCaseDemo {
    public static void main(String[] args) {
        String name = "JAVA";
        System.out.println(name.toLowerCase());
    }
}
```

Output

```
java
```

Example 7: substring()

```
public class SubStringDemo {
    public static void main(String[] args) {
        String str = "Programming";
        System.out.println(str.substring(3,8));
    }
}
```

Output

```
gramm
```

Example 8: replace()

```
public class ReplaceDemo {
   public static void main(String[] args) {
      String str = "Java";
      System.out.println(str.replace('a','o'));
   }
}
```

Output

Jovo

Example 9: trim()

```
public class TrimDemo {
   public static void main(String[] args) {
      String str = "   Java Programming   ";
      System.out.println(str.trim());
   }
}
```

Output

Java Programming

StringBuffer Class

Definition

StringBuffer is a predefined class used to create mutable strings.

Mutable means the existing object can be modified.

Syntax

```
StringBuffer sb = new StringBuffer("Java");
```

Characteristics of StringBuffer

- Mutable
- Faster for repeated modifications
- Supports append(), insert(), delete(), reverse(), replace(), etc.

StringBuffer Methods

Method	Description
append()	Adds text at end
insert()	Inserts text
replace()	Replaces characters
delete()	Deletes characters
reverse()	Reverses string
length()	Returns length
capacity()	Returns buffer capacity
charAt()	Returns character

Example 1: append()

```
public class AppendDemo {

   public static void main(String[] args) {

      StringBuffer sb = new StringBuffer("Java");

      sb.append(" Programming");

      System.out.println(sb);

   }

}
```

Output

```
Java Programming
```

Example 2: insert()

```
public class InsertDemo {

   public static void main(String[] args) {
```

```
        StringBuffer sb = new StringBuffer("Java");
        sb.insert(4," Language");
        System.out.println(sb);
    }
}
```

Output

```
Java Language
```

Example 3: replace()

```
public class ReplaceBuffer {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Java");
        sb.replace(0,4,"Python");
        System.out.println(sb);
    }
}
```

Output

```
Python
```

Example 4: delete()

```
public class DeleteDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Java Programming");
        sb.delete(4,16);
        System.out.println(sb);
    }
```

```
}
```

Output

```
Java
```

Example 5: reverse()

```
public class ReverseDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Java");
        sb.reverse();
        System.out.println(sb);
    }
}
```

Output

```
avaJ
```

Combined Practical (String + StringBuffer)

```
public class StringPractical {
    public static void main(String[] args) {
        // String Class
        String str = "Java Programming";
        System.out.println("Original String : " + str);
        System.out.println("Length : " + str.length());
        System.out.println("Upper Case : " + str.toUpperCase());
        System.out.println("Lower Case : " + str.toLowerCase());
        System.out.println("Character at Index 2 : " + str.charAt(2));
        System.out.println("Substring : " + str.substring(5,16));
```

```
        // StringBuffer Class

        StringBuffer sb = new StringBuffer("Java");

        System.out.println("\nOriginal Buffer : " + sb);

        sb.append(" Programming");

        System.out.println("Append : " + sb);

        sb.insert(5,"Language ");

        System.out.println("Insert : " + sb);

        sb.replace(0,4,"JAVA");

        System.out.println("Replace : " + sb);

        sb.delete(5,14);

        System.out.println("Delete : " + sb);

        sb.reverse();

        System.out.println("Reverse : " + sb);

        System.out.println("Length : " + sb.length());

        System.out.println("Capacity : " + sb.capacity());

    }

}
```

Wrapper Class in Java

Wrapper classes convert primitive data types into objects and objects back into primitive data types.

For example:

- int → Integer
- char → Character
- double → Double

Primitive Data Types and Wrapper Classes

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Example 1: Integer Wrapper Class

```
public class IntegerDemo {
   public static void main(String[] args) {
      Integer num = 100;
      System.out.println("Number = " + num);
   }
}
```

Output

```
Number = 100
```

Example 2: Double Wrapper Class

```
public class DoubleDemo {
   public static void main(String[] args) {
      Double value = 99.75;
      System.out.println(value);
   }
}
```

Output

```
99.75
```

Example 3: Character Wrapper Class

```
public class CharacterDemo {
   public static void main(String[] args) {
      Character ch = 'A';
      System.out.println(ch);
   }
}
```

Output

```
A
```

Example 4: Boolean Wrapper Class

```
public class BooleanDemo {
   public static void main(String[] args) {
      Boolean flag = true;
      System.out.println(flag);
   }
}
```

Output

```
true
```

Unboxing

Definition

Unboxing is the automatic conversion of a wrapper class object into its corresponding primitive data type.

```
public class UnBoxingDemo {
   public static void main(String[] args) {
      Integer obj = 200;
```

```
        int number = obj;
        System.out.println("Wrapper Object = " + obj);
        System.out.println("Primitive Value = " + number);
    }
}
```

Output

```
Wrapper Object = 200
Primitive Value = 200
```

Converting String to Integer

Example

```
public class ParseIntDemo {
    public static void main(String[] args) {
        String str = "500";
        int number = Integer.parseInt(str);
        System.out.println(number);
    }
}
```

Output

```
500
```